Week 3 — Design Studio: Answers

Design the support desk

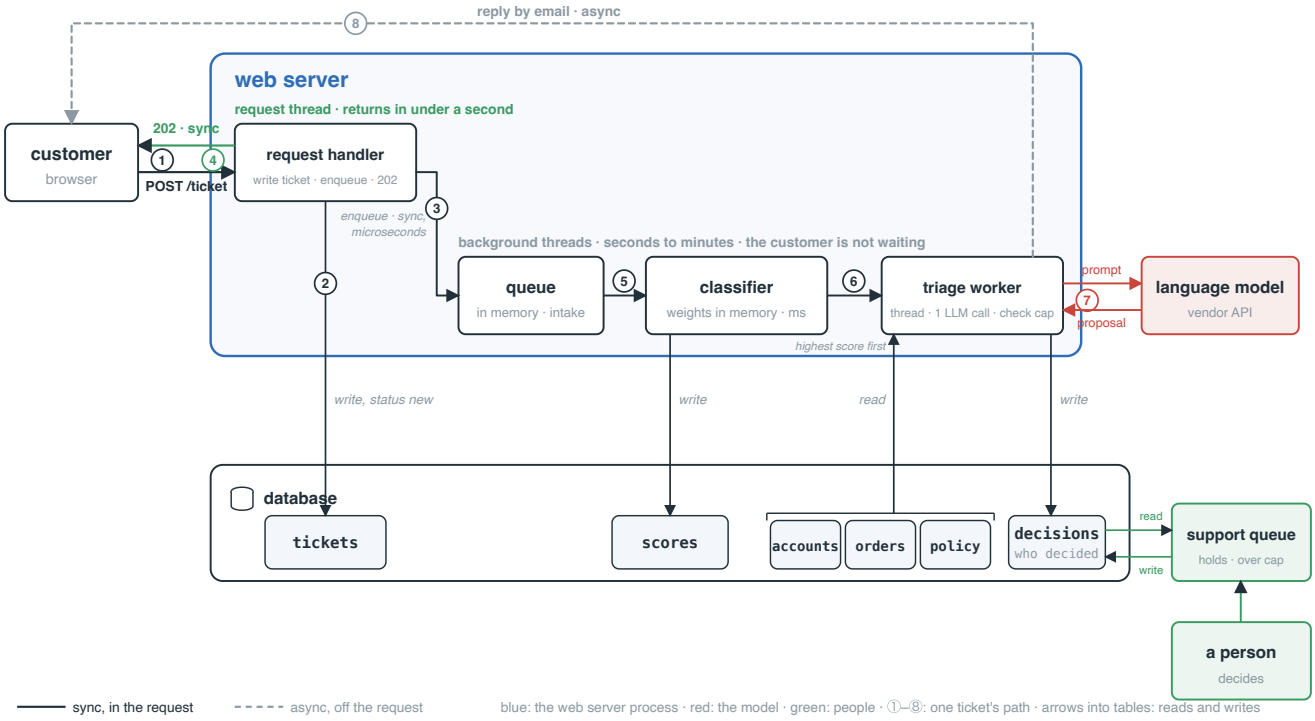
> Read this after the studio. It isn't the only good answer; it's what a strong answer contains. The system is the one the labs build, one component at a time; tonight's lab deploys the triage function.

---

1. The three numbers

NUMBER	VALUE	IMPLICATION
tickets per second, bad Monday	0.2	fine: nothing here needs to be fast
latency, confirmation	under a second	fine, as long as no model is in that path
latency, answer	within an hour	fine: the language model takes seconds, off the request
cost per month, classifier	negligible	fine
cost per month, language model	≈ $5 on a small model · ≈ $200 on a large one	fine: the support team's time is the real budget

2. The architecture



One web server process; every stage inside it. The request handler writes the ticket and returns the confirmation. The queue is in memory and comes first, so intake never waits on a model. The classifier is a weights file loaded into memory. The triage worker is a background thread that makes the one language-model call and checks the cap. Only the model, the database and the people are outside the process. At 0.2 tickets a second, one process is enough.

The flow of one ticket, numbered on the figure:

1. The customer's browser POSTs the ticket to the web server.
2. The request handler writes a `tickets` row with status new.
3. The handler pushes the ticket id onto the in-memory queue. Microseconds, still on the request thread.
4. The handler returns 202: *accepted*. The request is over, and the ticket is both recorded and queued before the customer hears so. Intake never depends on a model.
5. The classifier takes the next ticket off the queue, scores it, writes `scores`.
6. The triage worker takes the highest-scored waiting ticket and reads the three lookup tables: `accounts`, `orders` and `policy` (the cap and the allowed actions). The score exists only after step 5, so "highest score first" is a second, priority-ordered structure in memory; in a database it is an ORDER BY on the scored rows.
7. One language-model call: the model proposes; the worker checks the cap and writes `decisions`.

8. The worker emails the reply. The web server never connects back to a browser; the customer can also GET the ticket's status, which the handler reads from `decisions`.

The order of steps 2 to 4 is the point: persist, enqueue, then acknowledge. Because the row is written first, the in-memory queue can be rebuilt on a restart from every ticket whose status is still new. That is what makes this design survivable at all, and it is the same rule a distributed system follows with a queue service.

Holds, escalations and over-cap refunds are `decisions` rows too. The support queue is a view over them; a person's decision is written back to the same table, with who decided.

Six tables: `tickets` and `scores` are written by the stages that produce them; the three lookup tables `accounts`, `orders` and `policy` are read by the triage worker and written by nothing in this system; `decisions` is written by the worker and by people. The queue touches no table: it is in memory.

Two placements inside one process: the classifier runs off the queue, in milliseconds; the language-model call runs on a background thread, after the request has returned. The classifier's score decides which waiting ticket the worker takes next, so a bad Monday handles money first.

CODE DECIDES, FROM A TABLE	THE CLASSIFIER DECIDES	THE LANGUAGE MODEL DECIDES
the cap (`policy`), the allowed actions (`policy` × `accounts`), whether a refund is possible on this order (`orders`)	the score, 0 to 1	the action it proposes, the amount it proposes, the wording

The model proposes; code checks the proposal against the tables and disposes. A proposed refund over the cap goes to a person, whatever the model said.

3. The failure path *(if time)*

WHEN	WHAT RUNS
classifier down	tickets enter the queue unscored, in arrival order; nobody outside notices; scores backfilled later
language model down	the triage worker applies the rules alone: template tickets by rule, everything else to the support queue with the score beside it
both down	the support queue is the system; staff it

The customer sees the same confirmation in every case.

This design is brittle and does not scale well. We will be resolving resiliency and scaling in later sections.